VIT BHOPAL

NAME – ISHIKA AGGARWAL

REGISTRATION NUMBER – 25BCE10999

Course Faculty – M.K. Jayanti

BRANCH – COMPUTER SCIENCE (CORE)

PROJECT REPORT

1. Introduction

Email communications have become very popular in the modern digital age. Unfortunately, users often find themselves receiving many unsolicited or unknown emails, referred to as spam. Spam can be characterised as bulk emails that typically advertise non-existent products or services. Additionally, spam emails may contain misleading information and can include fraudulent emails that can be considered phishing.

Computer science and AI researchers continue to see spam detection as a highly valuable research topic. This project is going to build a very simple spam detection system that will be created in Python and will classify emails into one of two categories: spam or ham (non-spam) based on their contents.

The central objective of this project is to provide a better understanding of how text classification can be performed without the aid of other machine learning libraries and that this will be accomplished by manually coding the required logic.

2.Problem Statement

The rise in spam email is causing difficulties and possible use of unreliable information by individuals. I need to develop an automated means of identifying spam messages.

The task of this project is to establish a software solution that:

- Takes as input the body of the email to be analysed
- Examines the message's contents
- Assigns a class to the message (spam/not spam)
- Executes efficiently using standard computer programming methods

3. Functional Requirements

The system must fulfil the following functions:

- Accept email input from the user
- Preprocess the text (convert to lowercase, remove punctuation)
- Store and analyse word frequencies
- Train a classification model using sample data
- Predict whether the email is spam or ham
- Display the result clearly to the user
- Allow multiple inputs for repeated testing

4. Non-Functional Requirements

- **Performance:** The system should provide quick results with minimal delay.
- **Security:** The system should not store or share user data externally.
- **Usability:** The interface should be simple and easy to use.
- **Reliability:** The system should produce consistent results for similar inputs.
- **Scalability:** The system should allow future expansion with larger datasets.
- **Maintainability:** The code should be modular and easy to update.
- **Resource Efficiency:** The system should use minimal memory and processing power.

5. System Architecture

The system follows a simple pipeline structure:

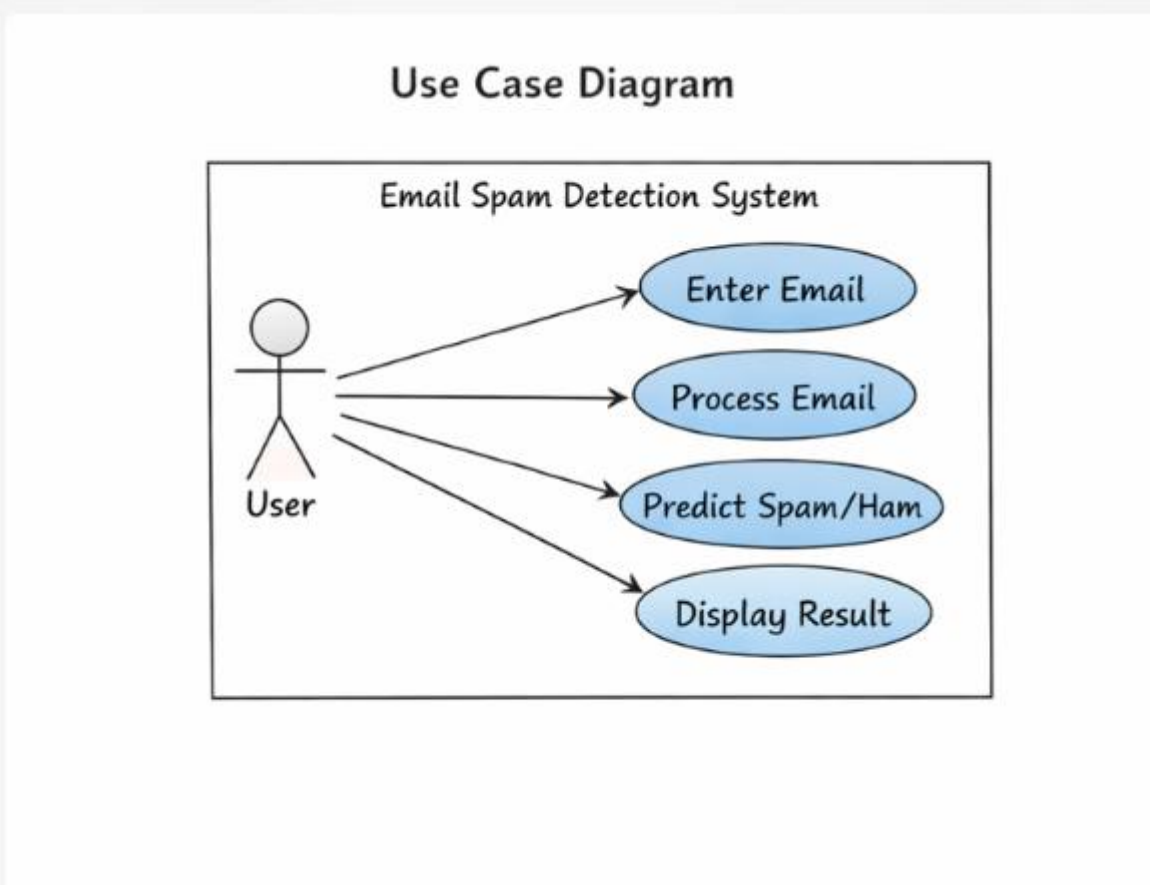
- User provides input
- Input is pre-processed
- Model analyses the processed data
- Prediction is generated
- Result is displayed

This modular structure ensures clarity and ease of understanding.

6. Design Diagrams

Use Case Diagram (Description)

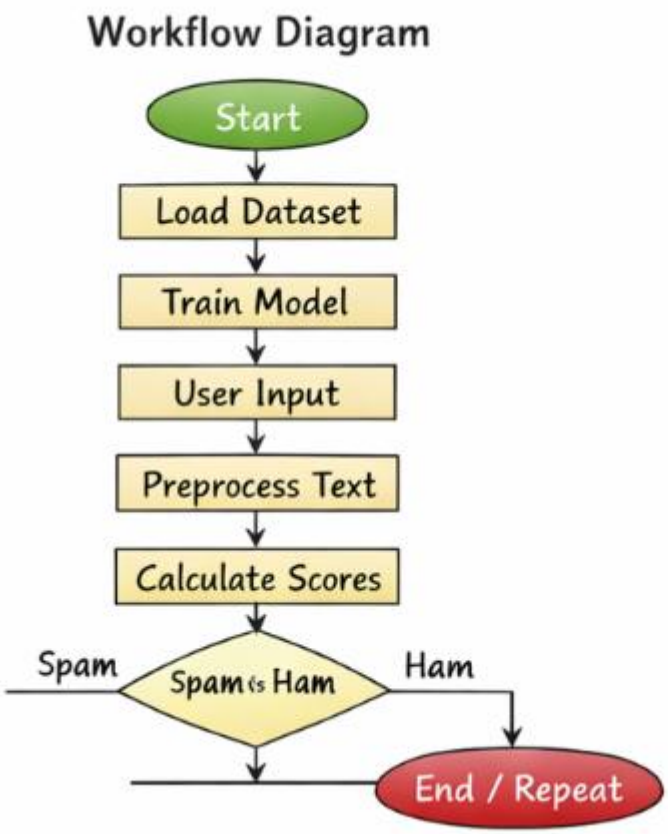
The user interacts with the system by entering an email. The system processes the input, predicts whether it is spam or not, and displays the result.



Workflow Diagram (Description)

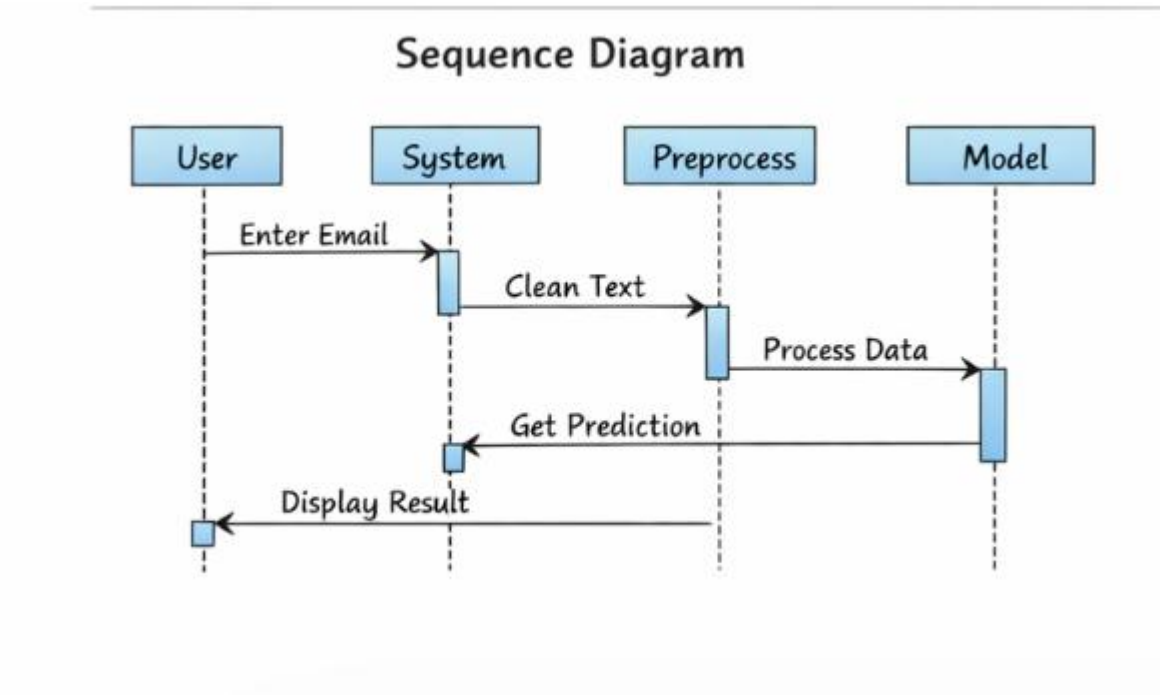
The workflow begins with loading the dataset and training the model. The user then enters an email, which is pre-processed and analysed. Based on the analysis,

the system compares spam and ham scores and displays the result.



Sequence Diagram (Description)

The user sends input to the system. The system preprocesses the text and sends it to the model. The model returns a prediction, which is then displayed to the user.



Class / Component Diagram (Description)

The system consists of different components such as:

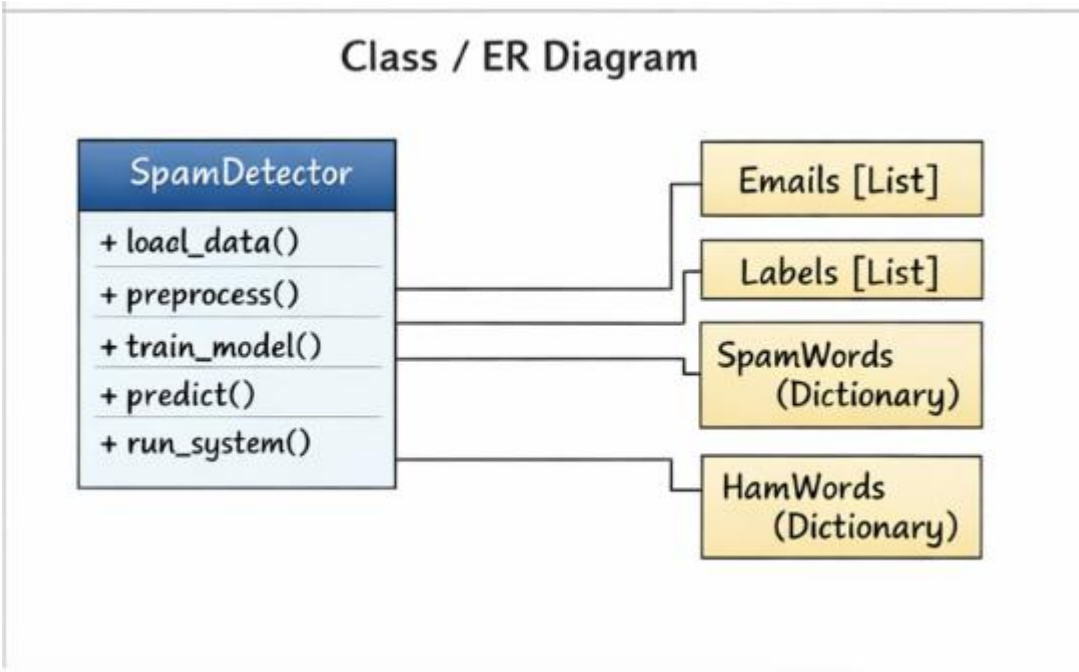
- Data handling module
- Preprocessing module
- Training module
- Prediction module
- User interface module

Each module performs a specific task, ensuring modularity.

ER Diagram (Description)

No external database is used.

- Dictionaries (for word frequency storage)



7. Design Decisions Rational

- The simple design makes the project easy to understand.
- Core logic of the project is developed without using external libraries.
- The chosen classification approach is a Naive Bayes-inspired approach, which has been shown to be a very effective way to classify text.
- An efficient way to store and look up word frequency is to use dictionaries.

8. Implementation Details

The project was implemented using Python, using the following steps:

* Loading a sample dataset.

* Preprocessing text (converting to lowercase and removing punctuation).

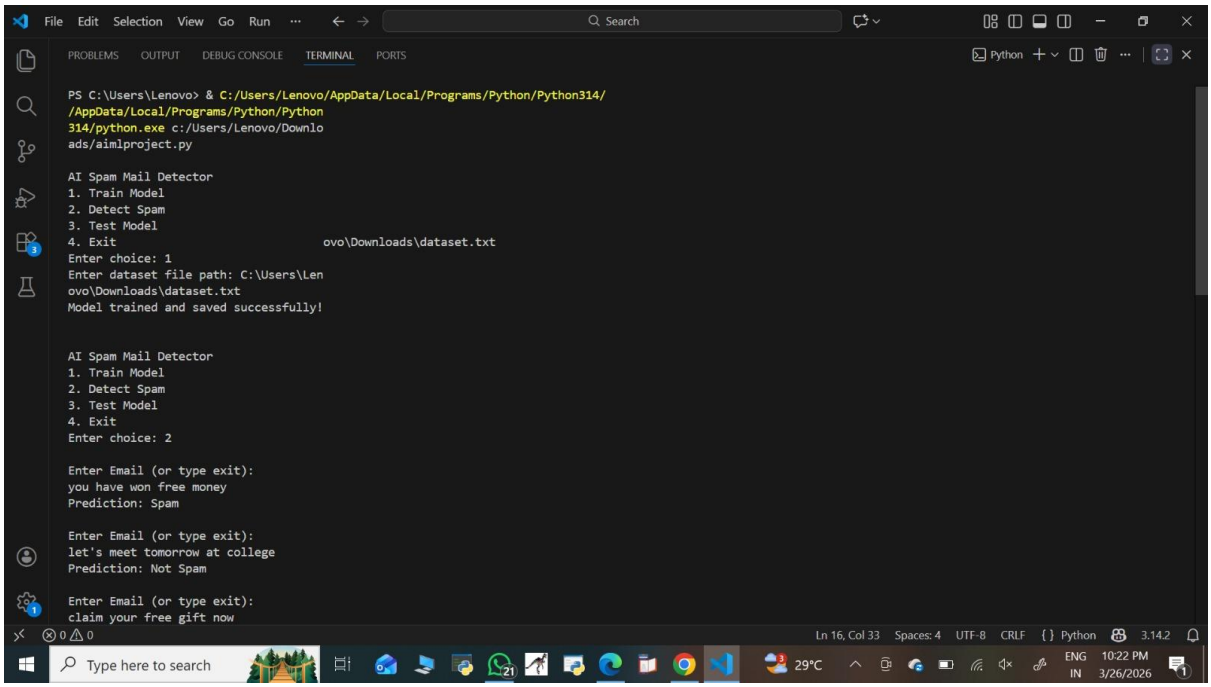
* Counting word frequencies for both spam and ham.

* Comparing the word frequency numbers (counts) to produce predictions.

* Displaying the results of the prediction.

The entire program is organized into functions for easier maintenance and readability.

9. Screenshot of Outputs



```
AI Spam Mail Detector
1. Train Model
2. Detect Spam
3. Test Model
4. Exit
Enter choice: 3
Enter test dataset path: C:\Users\Lenovo\Downloads\dataset.txt

Model Performance
Total Tested: 8
Correct Predictions: 8
Accuracy: 100.0 %

AI Spam Mail Detector
1. Train Model
2. Detect Spam
3. Test Model
4. Exit
Enter choice: 4
PS C:\Users\Lenovo>
```

```
4. Exit
Enter choice: 2

Enter Email (or type exit):
you have won free money
Prediction: Spam

Enter Email (or type exit):
let's meet tomorrow at college
Prediction: Not Spam

Enter Email (or type exit):
claim your free gift now
Prediction: Spam

Enter Email (or type exit):
call me when you are free
Prediction: Not Spam

Enter Email (or type exit):
exit

AI Spam Mail Detector
1. Train Model
2. Detect Spam
3. Test Model
4. Exit
Enter choice: 3
Enter test dataset path: C:\Users\Lenovo\Downloads\dataset.txt

Model Performance
Total Tested: 8
Correct Predictions: 8
```

10. Testing Methodology

Our testing process consists of both manual and automated.

Results of the program will be tested for consistency.

I will also review edge cases that are empty or mixed.

11. Difficulties I Encountered

- I implemented the classification process from scratch without a library.
- I wrote the algorithm to compare words for classification.
- I used a small sample size.
- I had to create a way to verify accuracy for a very simple algorithm.

12. What I Learned/Takeaways

- Learned how to preprocess text for classification.
- Learned how to detect and classify spam email.
- Learned how to manually implement a machine learning algorithm.
- Learned how to make an application modular.
- Improved our problem-solving skills.

13. Future Work

- I want to use a larger sample dataset for better accuracy.
- I want to incorporate a GUI interface.
- I want to integrate with email in real-time.
- I want to improve classification by using better algorithms.
- I want to be able to provide performance metrics on our application.

14. Source Materials Used

Official Python Documentation for reference.

Basic principles of machine learning.

Naive Bayes Algorithm (education sources)